Question Index - All 45 Questions

Q#	Question
Q1	What is artifact management? Why not store in Git?
Q2	Artifact types - JARs, Docker images, npm, Helm, PyPI
Q3	Repository types - hosted, proxy, group (virtual)
Q4	Nexus Repository basics - architecture, formats
Q5	Nexus repository management - creating repos, storage
Q6	Nexus user management - roles, privileges, LDAP/SSO
Q7	JFrog Artifactory basics - universal package manager
Q8	Artifactory repository types - local, remote, virtual
Q9	Artifactory package formats - Maven, Docker, Helm, npm
Q10	Harbor basics - cloud-native container registry
Q11	Harbor features - scanning, content trust, replication
Q12	Container registries - Docker Hub, ECR, GCR, ACR, GHCR
Q13	Semantic versioning - SNAPSHOT vs release, immutability
Q14	Artifact lifecycle - build, publish, promote, cleanup
Q15	Nexus in CI/CD - Maven, Docker, npm integration
Q16	Nexus cleanup policies - retention, blob store
Q17	Nexus proxy repositories - caching upstream
Q18	Nexus high availability - clustering, S3 backend
Q19	Artifactory in CI/CD - build info, promotion
Q20	Artifactory Xray - security scanning, impact analysis
Q21	Artifactory replication - multi-site, push/pull
Q22	Artifactory access control - permissions, tokens

Q23	Artifactory properties and metadata - AQL
Q24	Harbor vulnerability scanning - Trivy, policies
Q25	Harbor replication - between instances, to ECR/GCR
Q26	Harbor RBAC - projects, roles, robot accounts
Q27	Immutable artifacts - why never overwrite
Q28	Artifact promotion - dev to staging to prod
Q29	Dependency proxy/caching - resilience, speed
Q30	OCI registry standard - Helm, WASM, SBOMs
Q31	Image signing - cosign, Notary, admission webhooks
Q32	SBOM generation - CycloneDX, SPDX, syft
Q33	Nexus vs Artifactory vs Harbor - comparison
Q34	Artifact architecture - multi-region, DR, storage
Q35	Artifact retention strategy - compliance, cost
Q36	Supply chain security - SLSA, provenance
Q37	Migration between registries - zero-downtime
Q38	Cost optimization - tiering, cleanup, deduplication
Q39	Maven Central is down - builds failing
Q40	Docker Hub rate limit hit - CI throttled
Q41	Wrong artifact promoted to production
Q42	Nexus/Artifactory disk full
Q43	Vulnerability in base image across 50 services
Q44	Registry credentials leaked
Q45	Harbor replication lag - stale images

SECTION 1 - Junior Core Concepts (Q1–Q14)

Q1 What is artifact management? Why not store artifacts in Git?

The Simple Answer

An artifact is any output of a build process - compiled binaries (JARs, WARs), container images (Docker), packages (npm, PyPI, NuGet), Helm charts, Terraform modules, or documentation. Artifact management is the practice of storing, versioning, securing, and distributing these artifacts through a dedicated repository manager.

The Analogy - A Warehouse vs Your Desk

Git is your desk - great for source code (text files, small, diffable). An artifact repository is a warehouse - designed for large, binary, versioned packages. You would not store 500 MB Docker images on your desk, just like you would not store them in Git. The warehouse has inventory tracking, access control, temperature zones (different storage tiers), and a shipping dock (distribution). Your desk has none of that for large items.

Why Not Git?

Size - Git stores every version of every file. A 200 MB JAR committed 50 times = 10 GB of repository bloat. Artifact repos use deduplication and layer sharing (Docker images share base layers). Binary files - Git cannot diff binaries meaningfully. Every change stores a full copy. Artifact repos are optimized for binary storage. Access control - Git repos typically give read access to all files. Artifact repos provide fine-grained access: developers can pull dependencies but only CI can push builds. Metadata - Artifact repos attach metadata: build info, checksums, vulnerability scan results, promotion status. Git tracks file changes, not build provenance. Distribution - Artifact repos serve as download servers with CDN support, health checks, and pull-through caching. Git is not designed for high-throughput binary distribution.

Q2 Artifact types - JARs, Docker images, npm, Helm charts

The Simple Answer

Every language ecosystem produces different artifact types, and each has its own package format and repository protocol.

Java/JVM - JAR (library), WAR (web application), EAR (enterprise). Published to Maven repositories using Maven/Gradle. Identified by groupId:artifactId:version (com.example:my-app:2.1.0).

JavaScript/Node - npm packages (tarballs with package.json). Published to npm registries. Identified by @scope/name@version.

Python - Wheels (.whl) and source distributions (.tar.gz). Published to PyPI repositories. Identified by name==version.

Docker/OCI - Container images. Layers stored as blobs with manifests. Published to container registries (Docker Hub, ECR, Harbor). Identified by registry/repository:tag or @digest.

Helm - Chart packages (.tgz). Published to Helm repositories or OCI registries. Identified by chart-name:version.

Go - Module proxies. Go modules are downloaded from proxy.golang.org or private proxies.

Raw/Generic - Terraform providers, documentation, installers, firmware. Stored in generic/raw repositories.

Why this matters in interviews: "What artifact types does your team manage?" demonstrates real-world experience. Most DevOps teams manage at least Docker images, npm/PyPI packages, and Helm charts.

Q3 Repository types - hosted, proxy, group

The Simple Answer

Artifact repository managers organize storage into three repository types. Understanding these is fundamental to configuring any artifact manager.

Hosted - Your own artifacts. You build them, you push them here. Your application's JARs, your team's Docker images, your internal npm packages. You control the content completely.

Proxy (Remote) - A caching mirror of an external repository. When a developer requests a package from Maven Central or Docker Hub, the proxy fetches it, caches it locally, and serves future requests from cache. Benefits: faster builds (local cache), resilience (builds work even if the external repo is down), bandwidth savings, and security (scan cached artifacts before allowing use).

Group (Virtual) - A single URL that combines multiple repositories. A Maven group might include: your hosted releases repo + your hosted snapshots repo + your Maven Central proxy. Developers configure ONE repository URL and get artifacts from all sources transparently. Simplifies client configuration - one URL instead of three.

The Analogy - A Library System

A hosted repository is your personal bookshelf - your own books. A proxy repository is your local library's copy of books from the national library - they fetch and cache popular titles. A group repository is the library's catalog search - one search covers your shelf, the local library, and the national library simultaneously.

Q4 Nexus Repository basics - architecture, formats

The Simple Answer

Sonatype Nexus Repository Manager is one of the most widely used artifact managers. Available as OSS (free, community) and Pro (commercial with HA, RBAC, staging). Supports 20+ formats: Maven, npm, PyPI, Docker, Helm, NuGet, Go, raw, yum, apt, and more.

Architecture

Blob stores - Where binary content is physically stored. Default: file system. Pro: S3, Azure Blob. Each repository is assigned a blob store. Blob stores can be shared or dedicated.

Repositories - Logical containers with type (hosted/proxy/group) and format (maven/docker/npm). Each repository has its own URL.

Tasks - Scheduled operations: cleanup unused artifacts, rebuild indexes, compact blob stores, backup databases.

OrientDB (Nexus 3) - Internal database storing metadata, component info, and configuration. Not user-accessible. Backed up separately from blob stores.

Installation: Java-based. Runs on any OS with JVM. Docker deployment: docker run -d -p 8081:8081 sonatype/nexus3. Default admin password in /nexus-data/admin.password. Change immediately after first login.

Q5 Nexus repository management - creating repos, storage

The Simple Answer

Creating a repository in Nexus: Administration → Repositories → Create Repository → Select format (maven2, docker, npm) → Select type (hosted, proxy, group) → Configure storage (blob store), access (authentication), and format-specific settings.

Key Configuration

Docker hosted: Enable Docker registry API (HTTP connector on port 8082 or HTTPS). Configure Docker Bearer Token realm for authentication. Maven hosted: Choose release or snapshot policy. Release repos reject duplicate versions (immutable). Snapshot repos allow overwrites (mutable, for development). npm hosted: Standard npm registry protocol. Publish with npm publish --registry=http://nexus:8081/repository/npm-hosted/. Blob store assignment: Each repository uses a blob store. Separate blob stores for: Docker images (large, needs cleanup), Maven releases (permanent, rarely deleted), and temporary/snapshot artifacts (aggressive cleanup).

Q6 Nexus user management - roles, privileges, LDAP

The Simple Answer

Nexus controls access through: users (individual accounts), roles (collections of privileges), and privileges (specific permissions like "read from maven-releases" or "push to docker-hosted").

Key Patterns

Read-only role - Developers can pull artifacts but not push. All hosted repos readable. Proxy repos accessible. CI publisher role - CI/CD pipelines can push builds to specific hosted repos. Cannot delete or modify existing artifacts. Admin role - Full access. Repository creation, user management, configuration. Limit to 2-3 people.

LDAP/SSO Integration

Nexus Pro supports LDAP, SAML, and crowd authentication. Map LDAP groups to Nexus roles: the "developers" LDAP group gets the "read-only" Nexus role. The "ci-systems" group gets the "publisher" role. No local account management needed - all authentication through the organization's identity provider.

Q7 JFrog Artifactory basics - universal package manager

The Simple Answer

JFrog Artifactory calls itself the "universal artifact repository" - it supports every major package format in a single platform. Available as: OSS (free, limited formats), Pro (all formats, HA), Enterprise (multi-site, federation), and Cloud (SaaS). Artifactory is generally considered the most feature-rich artifact manager, with the deepest CI/CD integrations and the most advanced metadata capabilities.

Architecture

Storage backends - File system, S3, GCS, Azure Blob, or a database (for smaller deployments). Metadata stored in a database (PostgreSQL, MySQL, MSSQL, Oracle). Binary content stored in the configured storage backend with checksum-based deduplication - identical files are stored only once regardless of how many repositories contain them.

Checksum-based storage - Every artifact is stored by its SHA-256 checksum. If two repositories contain the same file, it is stored once. This dramatically reduces storage for organizations with many projects sharing common dependencies.

Q8 Artifactory repository types - local, remote, virtual

The Simple Answer

Artifactory uses different names for the same concepts as Nexus: Local = Hosted (your artifacts). Remote = Proxy (cached mirror of external repos). Virtual = Group (unified view of multiple repos).

Local Repositories

Created per package type and purpose: libs-release-local (production releases), libs-snapshot-local (development builds), docker-local (container images). Artifacts pushed by CI/CD after successful builds.

Remote Repositories

Point to external sources: Maven Central, npm registry, Docker Hub, PyPI.org. Artifactory caches downloaded artifacts. Configure cache expiry (how long before re-checking the upstream), missed cache expiry (how long to remember that an artifact does not exist upstream), and exclusion patterns.

Virtual Repositories

Combine local + remote repos into one URL. Example: libs-release virtual combines libs-release-local + jcenter-remote + maven-central-remote. Developers configure one URL in their build tool. Resolution order: local repos first, then remote repos.

Q9 Artifactory package formats

The Simple Answer

Artifactory supports 30+ package types natively. The most common in DevOps:

Maven - Java libraries and applications. Used with Maven, Gradle, Ant+Ivy. Docker - Container images. Artifactory acts as a Docker V2 registry. Helm - Kubernetes charts. Both Helm repository protocol and OCI format. npm - Node.js packages. Full npm registry compatibility. PyPI - Python packages. pip install from Artifactory. Go - Go modules. Acts as a Go module proxy. NuGet - .NET packages. Generic - Any file type. Used for: Terraform providers, compiled binaries, documentation archives, release artifacts. Debian/RPM - Linux packages. Manage internal deb/rpm repositories for apt/yum.

The value proposition: One platform for ALL artifact types. Instead of running: Docker Hub for images + npm registry for Node + PyPI server for Python + Helm repository for charts - Artifactory handles everything with unified access control, auditing, and replication.

Q10 Harbor basics - cloud-native container registry

The Simple Answer

Harbor is a CNCF graduated open-source container registry. Built on top of Docker Distribution (the open-source Docker registry), Harbor adds: vulnerability scanning, RBAC, replication, content trust (image signing), and audit logging. It is specifically designed for container images and OCI artifacts - not a general-purpose artifact manager like Nexus or Artifactory.

Architecture

Core - API server, authentication, authorization. Registry - Docker Distribution handling image push/pull. Database - PostgreSQL for metadata. Redis - Caching and job queue. Trivy - Integrated vulnerability scanner. Notary - Image signing (content trust). Job Service - Handles replication, scanning, and garbage collection asynchronously.

The Analogy

Harbor is a specialized container warehouse with built-in security guards (vulnerability scanning), identity checks (RBAC), and a shipping department (replication). Nexus and Artifactory are general-purpose warehouses that handle any type of goods. If you only manage containers, Harbor is purpose-built. If you manage containers AND Maven AND npm AND Helm, you need Nexus or Artifactory.

Q11 Harbor features - scanning, content trust, replication

Vulnerability Scanning

Every image pushed to Harbor is automatically scanned by Trivy. The scan report shows: CVE ID, severity (Critical, High, Medium, Low), affected package, and fixed version. Configure scan policies: block pulls of images with Critical vulnerabilities, or require manual approval.

Content Trust (Image Signing)

Harbor integrates with Notary for Docker Content Trust. Publishers sign images with their private key. Consumers verify signatures before pulling. Unsigned or tampered images are rejected. Ensures the image you pull is exactly what the publisher pushed - no man-in-the-middle modification.

Replication

Replicate images between Harbor instances or between Harbor and external registries (ECR, GCR, ACR, Docker Hub). Push-based (Harbor pushes to remote) or pull-based (Harbor pulls from remote). Trigger: manual, scheduled, or event-based (on push). Used for: multi-region deployment, disaster recovery, migrating between registries.

Q12 Container registries - Docker Hub, ECR, GCR, ACR, GHCR

The Simple Answer

Registry	Provider	Free Tier	Private Repos	Scanning	Best For
Docker Hub	Docker	1 private repo	Paid plans	Basic	Public images, open source
ECR	AWS	500 MB/month	Unlimited	Enhanced (Inspector)	AWS-native workloads
GCR/Artifact Registry	GCP	Per-project	Unlimited	On-demand	GCP workloads
ACR	Azure	Basic tier	Unlimited	Defender for Cloud	Azure workloads
GHCR	GitHub	Unlimited public	Unlimited private	Dependabot	GitHub-hosted projects
Harbor	Self-hosted	Unlimited	Unlimited	Trivy built-in	Self-hosted, multi-cloud
Quay	Red Hat	Unlimited public	Paid	Clair	OpenShift environments

Key Decision Factors

If you are all-in on one cloud, use the native registry (ECR, GCR, ACR) - tightest IAM integration, lowest latency, simplest authentication. If you are multi-cloud or hybrid, use Harbor (self-hosted, vendor-neutral). If you need a universal artifact manager (not just containers), use Artifactory or Nexus.

Q13 Semantic versioning - SNAPSHOT vs release

The Simple Answer

Semantic Versioning (semver): MAJOR.MINOR.PATCH. 2.1.3 means: 2 = breaking changes since 1.x, 1 = new features since 2.0, 3 = bug fixes since 2.1.0. Consumers know: upgrading 2.1.3 → 2.1.4 is safe (patch). 2.1.x → 2.2.0 adds features (minor). 2.x → 3.0 may break things (major).

SNAPSHOT (Java/Maven): A mutable, in-development version. 2.1.0-SNAPSHOT is "working toward 2.1.0 release." SNAPSHOTs can be overwritten - the latest build replaces the previous. Used during active development. Never deploy SNAPSHOTs to production.

Release: An immutable, published version. 2.1.0 is final. Once published, it cannot be changed - if you need a fix, publish 2.1.1. Release repositories reject duplicate versions to enforce immutability. This guarantee is fundamental - a consumer pulling version 2.1.0 today gets exactly the same artifact as pulling 2.1.0 next year.

The golden rule: Never overwrite a released artifact. If you made a mistake, publish a new version. Overwriting breaks build reproducibility - someone's build worked yesterday with version 2.1.0 but fails today because you changed 2.1.0's content. This is why release repositories are configured as immutable.

Q14 Artifact lifecycle - build, publish, promote, cleanup

The Simple Answer

Build - CI compiles code, runs tests, produces the artifact (JAR, Docker image, npm package). The artifact is tagged with a version (from Git tag, commit SHA, or semver).

Publish - CI pushes the artifact to a development/snapshot repository. The artifact is stored but not yet approved for production.

Promote - After testing, the artifact is promoted (copied or moved) from the dev repository to the staging repository, then to the production repository. The same binary moves through environments - no rebuilding. What you tested is what you deploy.

Distribute - Production systems pull from the production repository. CDN/edge caching for global distribution. Consumers (other teams, customers) access the artifact.

Cleanup - Old, unused artifacts are deleted by retention policies. SNAPSHOT artifacts older than 30 days. Development builds not promoted within 90 days. Keep all production releases for compliance (or per retention policy).

SECTION 2 - Mid-Level Production Patterns (Q15–Q32)

Q15 Nexus in CI/CD - Maven, Docker, npm

Maven Integration

Configure settings.xml with Nexus credentials. Maven resolves dependencies from Nexus (proxy repos). mvn deploy pushes built artifacts to Nexus (hosted repo). Gradle uses the maven-publish plugin with Nexus repository URLs. In CI: credentials come from pipeline secrets, never hardcoded.

Docker Registry

Nexus acts as a Docker V2 registry. docker login nexus.example.com:8082. docker push nexus.example.com:8082/my-app:v2.1.0. Configure a Docker proxy repo pointing to Docker Hub - all base image pulls are cached locally.

npm Integration

npm config set registry http://nexus.example.com:8081/repository/npm-group/. npm publish to the hosted repo. npm install resolves from the group repo (hosted + proxy). .npmrc file in the project configures authentication.

The CI/CD pattern: Build → publish to dev repo → run tests → promote to staging repo → integration tests → promote to prod repo → deploy from prod repo. Each promotion is a copy, not a rebuild. The artifact binary is identical at every stage.

Q16 Nexus cleanup policies - retention, blob store

The Simple Answer

Without cleanup, Nexus grows indefinitely. Docker images, SNAPSHOT artifacts, and unused dependencies accumulate. Cleanup policies define what to delete and when.

Cleanup Policies

Create a policy: Administration → Repository → Cleanup Policies. Criteria: last downloaded more than X days ago, last published more than X days ago, regex matching component name/version, or release type (SNAPSHOT only). Attach the policy to specific repositories. Schedule the cleanup task: Administration → System → Tasks → Create "Cleanup repositories using their associated policies."

Blob Store Compaction

Deleting artifacts from repositories does NOT immediately free disk space. Deleted artifacts are marked for removal in the blob store. Run the "Compact blob store" task to reclaim disk space. Schedule this after cleanup runs.

Best practice: Aggressive cleanup for snapshots (delete after 30 days). Moderate cleanup for development

builds (90 days). Never auto-delete production releases (keep for compliance). Run cleanup weekly, compaction daily. Monitor disk usage with alerts at 80% capacity. See scenario Q42.

Q17 Nexus proxy repositories - caching upstream

The Simple Answer

Proxy repositories cache artifacts from external sources. First request for a package from Maven Central: Nexus fetches from Maven Central, caches locally, serves to the developer. Subsequent requests: served from cache instantly. The external source is never contacted again until the cache expires.

Why Proxy Repos Are Essential

Speed - Local cache serves in milliseconds vs hundreds of milliseconds from Maven Central. For a build with 200 dependencies, this saves minutes. Resilience - If Maven Central, Docker Hub, or npm registry goes down, builds continue from cache. Your team's productivity is not dependent on external service availability. Security - All external artifacts pass through your proxy. You can scan them for vulnerabilities before they reach developers. You can block specific packages or versions. Bandwidth - 50 developers pulling the same 500 MB base image from Docker Hub = 25 GB of bandwidth. With a proxy, it is downloaded once = 500 MB. Rate limits - Docker Hub limits anonymous pulls to 100/6 hours and authenticated to 200/6 hours. A proxy makes one pull and caches it - unlimited internal pulls. See scenario Q40.

Q18 Nexus high availability - clustering

The Simple Answer

Nexus Pro supports active-active clustering. Multiple Nexus instances share the same blob store (S3, Azure Blob) and database. A load balancer distributes requests. If one node fails, others continue serving. Benefits: horizontal scalability for read-heavy workloads, zero-downtime upgrades, and disaster recovery.

Storage Backends for HA

S3 blob store - Store blobs in AWS S3. Unlimited storage, built-in replication, 99.999999999% durability. Nexus nodes are stateless - they read/write to S3. Easy to scale nodes independently. Database - Shared PostgreSQL (Nexus 3.x Pro) or the internal OrientDB in replicated mode. PostgreSQL is recommended for production HA - well-understood, easy to backup, supports replication.

Q19 Artifactory in CI/CD - build info, promotion

Build Info

Artifactory captures comprehensive build metadata: which source commit triggered the build, which dependencies were resolved, which artifacts were produced, build environment variables, and test results. This build info is attached to the artifacts and stored permanently. "Show me exactly what went into building version 2.1.0" - Artifactory has the answer.

Build Promotion

Promote a build from dev to staging to production with a single API call. The artifacts are copied (or moved) between repositories. The build info is updated with the promotion status. Promotion can require approval (manual gate) or be automated (after tests pass).

Release Bundles (Enterprise)

Package multiple artifacts into a signed, immutable release bundle. Distribute the bundle to multiple Artifactory instances (multi-site). The entire bundle is verified as a unit - if any artifact is tampered with, the bundle verification fails. Used for: distributing software releases to edge locations, customers, or air-gapped environments.

Q20 Artifactory Xray - security scanning

The Simple Answer

JFrog Xray is a security scanning tool deeply integrated with Artifactory. It scans artifacts for: known vulnerabilities (CVEs), license compliance violations, and operational risk. Unlike standalone scanners that check images at one point in time, Xray continuously monitors - when a new CVE is published, Xray immediately identifies all affected artifacts in your repositories.

Impact Analysis

When a new CVE is announced, Xray traces the impact: "CVE-2024-1234 affects log4j 2.14.1. These 23 artifacts in your repositories depend on log4j 2.14.1. These 8 builds deployed to production include this dependency." This impact analysis is Xray's killer feature - answering "how bad is this?" in seconds instead of hours of manual investigation.

Policies and Watches

Policies define rules: block download of artifacts with Critical CVEs, send alert on High CVEs, require manual approval for artifacts with GPL licenses. Watches apply policies to repositories or builds. A watch on the production Docker repo blocks pulling any image with Critical vulnerabilities - the deployment pipeline fails before the vulnerable image reaches production.

Q21 Artifactory replication - multi-site

The Simple Answer

Replication copies artifacts between Artifactory instances across data centers, regions, or clouds. Push replication - Source pushes artifacts to the target. Event-based (on every push) or scheduled. Pull replication - Target pulls artifacts from the source. Useful when the target is behind a firewall. Multi-push - One source replicates to many targets simultaneously. Used for global distribution.

Use Cases

Multi-region deployment - Artifacts replicated to registries in each region. Production clusters pull from the local registry - low latency, no cross-region traffic. Disaster recovery - Replicate to a standby instance in another region. If the primary fails, switch DNS to the standby. Edge deployment - Replicate specific artifacts to edge locations for IoT or retail deployments.

Q22 Artifactory access control

The Simple Answer

Permissions - Fine-grained: read, deploy (push), delete, annotate (add metadata), manage (admin). Applied to: specific repositories, repository patterns (docker-*), or paths within repositories. Groups - Collections of users. Map to LDAP/SSO groups. Assign permissions to groups, not individuals. Access tokens - Scoped tokens for CI/CD and automation. Short-lived. Replaceable without changing user passwords. Federated repositories - In multi-site deployments, federated repos synchronize access control and content across all sites. A push to any site replicates to all others automatically.

Q23 Artifactory properties and metadata

The Simple Answer

Artifactory allows attaching arbitrary key-value properties to any artifact. This is metadata that describes the artifact beyond its version number.

Examples

build.number=142, build.timestamp=2025-05-15, git.commit=abc1234, tested=true, promoted.to=staging, approved.by=alice, scan.status=clean. Properties enable: searching ("find all artifacts promoted to production this week"), automation ("only deploy artifacts with tested=true"), and auditing ("who approved this release?").

AQL (Artifactory Query Language)

Query artifacts by properties, build info, or any metadata: items.find({"@promoted.to": "production", "@build.number": {"$gt": "100"}}). Powerful for: finding all production artifacts, cleanup automation, compliance reporting, and vulnerability impact analysis.

Q24 Harbor vulnerability scanning

The Simple Answer

Harbor integrates Trivy as its default scanner. Every image pushed to Harbor is scanned automatically. The scan report shows: each CVE with severity, affected package name and version, and fixed version (if available).

Scan Policies

Prevent vulnerable images from being pulled - Set a severity threshold: block pulls of images with any Critical or High CVEs. The pull fails with a clear error message. Developers must fix vulnerabilities before the image can be deployed. CVE allowlist - Known false positives or accepted risks can be allowlisted per project. Allowlisted CVEs do not block pulls. Document the justification for each allowlisted CVE.

Scan-on-Push vs Scheduled

Scan-on-push (default) - Immediate feedback. Developers know about vulnerabilities within minutes of pushing. Scheduled scan - Re-scan existing images periodically. Catches newly discovered CVEs in already-stored images. Essential because a clean image today may have a Critical CVE tomorrow when a new vulnerability is published.

Q25 Harbor replication

The Simple Answer

Harbor replicates images to and from: other Harbor instances, Docker Hub, ECR, GCR, ACR, Quay, Artifactory, GitLab Registry, and any Docker V2 registry.

Replication Rules

Define: source (registry, project, repository filter), destination (target registry, namespace), trigger (manual, scheduled, event-based). Filters: replicate only images matching specific tags (v*, release-*), or specific repositories.

Multi-Region Pattern

Production Harbor in us-east-1. Replicate to Harbor instances in eu-west-1 and ap-southeast-1. Kubernetes clusters in each region pull from the local Harbor - low latency, no cross-region data transfer costs. Event-based replication ensures images are available in all regions within seconds of push.

Q26 Harbor RBAC

The Simple Answer

Harbor organizes access around projects. Each project has its own repositories, members, and policies. Roles: Project Admin - Full control over the project. Developer - Push and pull images. Guest - Pull only. Robot accounts - Service accounts for CI/CD with scoped, time-limited tokens. No interactive login. Cannot manage project settings.

OIDC Integration

Harbor integrates with OIDC providers (Keycloak, Okta, Azure AD). Users log in with SSO. OIDC groups map to Harbor roles. No local account management - all through the identity provider.

Q27 Immutable artifacts - why never overwrite

The Simple Answer

An immutable artifact cannot be modified or replaced once published. Version 2.1.0 always returns the exact same binary. This guarantee is the foundation of reproducible builds and reliable deployments.

Why Immutability Is Non-Negotiable

Build reproducibility - If version 2.1.0 changes, builds that depend on it may break without any code change. "It worked yesterday" becomes impossible to investigate. Deployment safety - Rolling back to "version 2.1.0" must produce the exact same application. If 2.1.0 was overwritten, the rollback deploys different code. Audit trail - Compliance requires knowing exactly what was deployed. If artifacts are mutable, audit evidence is unreliable. Trust - Teams must trust that pulling a version gives them the tested, approved artifact. Mutability breaks that trust.

Implementation

Container images: Reference by digest (@sha256:abc123) not tag (:latest). Tags are mutable pointers. Digests are content-addressed - the hash changes if any byte changes. Maven: Configure hosted repos with "Release" policy - reject duplicate versions. npm: npm publish refuses to overwrite an existing version by default. Helm: Use --version strictly. Never reuse a chart version.

The :latest anti-pattern: Never deploy :latest to production. It points to whatever was pushed most recently - you have no idea which version is running. Use specific version tags (v2.1.0) or digests. :latest is acceptable only for development and local testing.

Q28 Artifact promotion - dev to staging to prod

The Simple Answer

Promotion moves an artifact between repositories representing different maturity levels, without rebuilding. The same binary artifact that was tested in staging is the exact binary deployed to production.

The Promotion Pipeline

Step 1 - Build and publish. CI builds the artifact, pushes to the dev repository. Tags: build number, commit SHA. Step 2 - Test and promote to staging. Integration tests pass. The artifact is copied (not rebuilt) from dev to staging repo. In Nexus: REST API copy. In Artifactory: promote build API. In Harbor/registries: Skopeo copy or registry replication. Step 3 - Approve and promote to production. Manual approval gate (PR, Jira ticket, Slack approval). The artifact is copied from staging to prod repo. Step 4 - Deploy from production repo. Kubernetes pulls from the prod registry. Only the prod registry is accessible from the production cluster network.

Why Not Rebuild

Rebuilding for each environment introduces risk. Even with the same Dockerfile and source code, a rebuild may produce a different binary: base image tags may have updated (ubuntu:22.04 changed since last build), dependency versions may have changed (npm install pulls newer patch versions), build tools may have updated. Promotion guarantees binary identity across environments.

Q29 Dependency proxy/caching

The Simple Answer

A dependency proxy sits between your developers/CI and external package registries. It caches downloaded packages and serves subsequent requests from cache.

Benefits Beyond Speed

Availability - npm registry outage? Your proxy has cached all your project's dependencies. Builds continue uninterrupted. Security scanning - Scan cached packages before allowing consumption. Block known malicious packages (typosquatting, supply chain attacks). License compliance - Track which open-source licenses are used across all projects. Block packages with incompatible licenses. Bandwidth and cost - Docker Hub rate limits (100 pulls/6 hours anonymous). A proxy makes one pull and serves unlimited internal requests from cache. AWS NAT Gateway charges per GB - caching reduces cross-internet traffic significantly.

Q30 OCI registry standard

The Simple Answer

The OCI Distribution Specification defines how container registries store and serve content. Originally for Docker images, OCI registries now store any content type: Helm charts, WebAssembly modules, SBOMs, signatures, and arbitrary files. This "OCI artifacts" capability means a single registry can replace multiple purpose-built stores.

Practical Impact

Helm charts in OCI: helm push mychart-1.0.0.tgz oci://registry.example.com/charts/. No separate ChartMuseum needed. Signatures in OCI: cosign sign and cosign verify store signatures as OCI artifacts alongside the image. SBOMs in OCI: Attach a CycloneDX or SPDX SBOM to an image as an OCI artifact. The SBOM travels with the image - wherever the image goes, its bill of materials follows.

Q31 Image signing and verification - cosign, Notary

The Simple Answer

Image signing cryptographically proves who built an image and that it has not been tampered with since signing.

cosign (sigstore) - The modern standard. Keyless signing using OIDC identity (GitHub Actions, Google Cloud Build). Signs images and stores signatures in the OCI registry alongside the image. Verify: cosign verify --certificate-identity=ci@example.com --certificate-oidc-issuer=https://token.actions.githubusercontent.com registry.example.com/app:v2.1.0. No key management - identity-based signing.

Notary v2 (notation) - Docker Content Trust v2. Plugin-based signing. Supports multiple signing key types. Used by Harbor for content trust.

Enforcement

Kubernetes admission webhook (Kyverno, OPA Gatekeeper, Connaisseur) rejects Pods that reference unsigned images. Every image in production must be signed by the CI pipeline - no manually built or unsigned images allowed.

Q32 SBOM generation and storage

The Simple Answer

An SBOM (Software Bill of Materials) lists every component in a software artifact - libraries, frameworks, base image packages, and their versions. If a vulnerability is discovered in a dependency, the SBOM instantly answers: "Which of my artifacts are affected?"

Formats

CycloneDX - OWASP standard. JSON/XML. Includes: components, dependencies, vulnerabilities, licenses. Widely adopted in DevSecOps. SPDX - Linux Foundation standard. Focus on license compliance. Used in legal and compliance contexts.

Generating SBOMs

syft (Anchore) - Generate SBOMs from container images, directories, or archives: syft registry.example.com/app:v2.1.0 -o cyclonedx-json > sbom.json. Trivy - trivy image --format cyclonedx registry.example.com/app:v2.1.0. Docker - docker sbom (experimental, uses syft internally). Attach the SBOM to the image as an OCI artifact: cosign attach sbom --sbom sbom.json registry.example.com/app:v2.1.0.

SECTION 3 - Senior Architecture (Q33–Q38)

Q33 Nexus vs Artifactory vs Harbor - comparison

The Simple Answer

Feature	Nexus OSS	Nexus Pro	Artifactory	Harbor
Price	Free	$$$	$$$$	Free
Formats	20+	20+	30+	Container only
HA	No	Yes	Yes	Yes
Replication	No	Yes	Advanced	Yes
Vulnerability scan	No	Limited	Xray (powerful)	Trivy (built-in)
Build info	No	Yes	Deep integration	No
Best for	Small teams, tight budget	Enterprise, multi-format	Enterprise, full lifecycle	Container-only, multi-cloud

Decision Framework

Budget-constrained, multi-format: Nexus OSS. Free. Supports all major formats. No HA - manage with backups and fast recovery. Enterprise, full artifact lifecycle: Artifactory. Best build info, promotion, replication, and scanning (Xray). Worth the cost for large organizations. Container-only, self-hosted: Harbor. Purpose-built. Free. Excellent scanning and RBAC. CNCF graduated. Cloud-native, single cloud: Use the cloud provider's registry (ECR, GCR, ACR) - tightest integration, simplest auth. Add Nexus or Artifactory for non-container artifacts.

Q34 Artifact management architecture - multi-region, DR

Multi-Region Architecture

Primary artifact manager in the main region. Replicas in each deployment region. CI pushes to the primary. Replication distributes to replicas. Production clusters pull from the local replica - low latency, no cross-region traffic, resilience if the primary is unavailable.

Disaster Recovery

Active-passive: Primary serves all traffic. Standby receives replication. DNS failover switches to standby. RPO depends on replication lag (seconds for event-based, minutes for scheduled). Active-active: Both sites serve traffic. Bidirectional replication. Conflict resolution for simultaneous pushes (last-write-wins or primary-wins). More complex but zero-downtime failover.

Storage Backend Selection

S3 with cross-region replication for AWS deployments - storage is automatically replicated. GCS multi-region buckets for GCP. Azure Blob with geo-redundant storage. The storage backend handles data durability - the artifact manager handles the application layer.

Q35 Artifact retention strategy

The Simple Answer

Retention balances: compliance requirements (keep for X years), cost (storage is not free), and operational need (keep what is deployable, delete what is not).

Retention Tiers

Production releases: Keep forever (or per compliance: 7 years for finance, 10 years for healthcare). These are the artifacts that were deployed to production. You must be able to reproduce any past production state. Staging/QA artifacts: Keep for 90 days. These are tested but not promoted to production. If they are not promoted within 90 days, they will never be. Development/SNAPSHOT artifacts: Keep for 30 days. Actively changing. Old snapshots have no value. Aggressive cleanup. Base images and dependencies (proxy cache): Keep as long as they are actively used. Delete when not downloaded for 180 days. Can always re-fetch from upstream if needed.

Legal Hold

Some artifacts may be subject to legal hold - litigation or regulatory investigation requires preserving specific versions. Legal hold overrides normal retention policies. Tag legal-hold artifacts with metadata and exclude them from cleanup.

Q36 Supply chain security - SLSA, provenance

The Simple Answer

Software supply chain attacks compromise the build process or dependencies to inject malicious code. SolarWinds (2020), Log4Shell (2021), and Codecov (2021) demonstrated the devastating impact. Supply chain security ensures: the source code is what you think it is, the build process was not tampered with, and the artifact is exactly what the build produced.

SLSA Framework (Supply-chain Levels for Software Artifacts)

Level 1 - Build process is documented. Provenance (who built what, from which source) is generated. Level 2 - Build runs on a hosted service (not a developer's laptop). Provenance is signed. Level 3 - Build runs on a hardened, isolated build platform. Source and build platform are verified. Provenance is non-forgeable. Level 4 - Two-party review of all changes. Hermetic, reproducible builds. Full provenance chain from source to deployed artifact.

Implementation

Provenance: Generate SLSA provenance attestations in CI (GitHub Actions has built-in SLSA provenance). Attach to artifacts as OCI artifacts (cosign attest). Verification: Kubernetes admission webhooks verify provenance

before allowing deployment. "Only deploy artifacts built by our CI system from our repository." Dependency pinning: Pin all dependency versions with checksums. Detect if an upstream package changes without a version bump (a sign of compromise).

Q37 Migration between registries

The Simple Answer

Migrating from one artifact manager to another (Nexus to Artifactory, Docker Hub to Harbor) requires moving potentially terabytes of artifacts without disrupting ongoing builds and deployments.

Strategy

Phase 1 - Dual-write. CI pushes to both old and new registries. New artifacts exist in both. Ongoing. Phase 2 - Migrate history. Copy existing artifacts from old to new. For container images: Skopeo sync (registry-to-registry copy without local pull). For Maven/npm: custom scripts using repository APIs. For large migrations: use the new registry's import feature if available. Phase 3 - Switch consumers. Update Kubernetes image references, build tool configurations, and developer settings to point to the new registry. Do this gradually - one team or cluster at a time. Phase 4 - Decommission old. After all consumers use the new registry and a stability period (30 days), shut down the old registry.

Zero-Downtime Pattern

Run both registries in parallel throughout the migration. The new registry proxies the old one during transition - requests for artifacts not yet migrated are fetched from the old registry and cached in the new one. Consumers switch to the new registry immediately. Migration happens transparently in the background.

Q38 Cost optimization

The Simple Answer

Artifact storage grows continuously. Without optimization, costs increase linearly (or worse) over time.

Strategies

Cleanup automation - Delete unused artifacts on schedule. The single biggest cost saver. Most organizations store 10x more artifacts than needed. Storage tiering - Hot storage for frequently accessed artifacts (last 90 days of builds). Cold storage (S3 Glacier, Azure Cool) for compliance-retained old artifacts. Deduplication - Artifactory's checksum-based storage deduplicates automatically. Docker layer sharing in any registry reduces storage (10 images sharing the same base layer store the base once). Minimize image sizes - Distroless and Alpine base images (20 MB) instead of Ubuntu (77 MB). Multi-stage builds exclude build tools from the final image. Smaller images = less storage, faster pulls, lower bandwidth costs. Registry-per-region vs shared - A registry in each region duplicates storage but reduces cross-region bandwidth (which is expensive). Calculate: is the replication storage cost less than the bandwidth cost? Usually yes for frequently pulled images.

SECTION 4 - Tricky Scenarios (Q39–Q45)

Q39 Maven Central is down - builds failing

What Happened

Maven Central is unreachable. All Maven/Gradle builds that need to download dependencies fail. 50 developers are blocked.

Step-by-Step

Step 1 - If you have a proxy repository: Builds should continue from cache. Check: is the proxy repo healthy? Can it serve cached artifacts? If the proxy is configured with "block outbound" on upstream failure, disable it temporarily - serve from cache only.

Step 2 - If you do NOT have a proxy: This is why you need one. Developers with local .m2/repository caches can continue building. Share a developer's cache as a temporary file-based repo: mvn install:install-file for critical missing artifacts.

Step 3 - Check Maven Central status. status.maven.org or search Twitter/X for reports. If it is a CDN issue, try alternative mirrors: repo1.maven.org, maven.google.com (for Android/Google libraries).

Prevention

Set up a Nexus or Artifactory proxy for every external dependency source (Maven Central, npm, Docker Hub, PyPI). Pre-warm the cache by building all projects once through the proxy. Verify cache hit rates regularly - 90%+ means you are well-protected. This is not optional for any serious engineering organization.

Q40 Docker Hub rate limit hit - CI throttled

What Happened

CI pipelines fail with: "You have reached your pull rate limit." Docker Hub limits: anonymous 100 pulls/6 hours, authenticated 200 pulls/6 hours. With 20 CI pipelines running 10 builds/hour, each pulling a base image, you hit the limit in under 2 hours.

Step-by-Step

Step 1 - Immediate fix. Authenticate all Docker Hub pulls (doubles the limit to 200). Create a Docker Hub service account. Configure CI runners with docker login.

Step 2 - Set up a Docker proxy. Nexus or Artifactory proxy pointing to Docker Hub. All pulls go through the proxy. The proxy fetches from Docker Hub once and caches. Unlimited internal pulls from cache. Rate limit impact: reduced to the number of unique images pulled, not total pulls.

Step 3 - Mirror critical base images. Copy frequently used base images (ubuntu, alpine, nginx, python, node) to your private registry. Update Dockerfiles to pull from your registry: FROM registry.example.com/mirrors/ubuntu:22.04 instead of FROM ubuntu:22.04.

Step 4 - Docker Hub Pro/Team. $5-9/month per user. 5000-50000 pulls/day. For organizations with heavy usage, the subscription is cheaper than the engineering time lost to rate limits.

Q41 Wrong artifact promoted to production

What Happened

A CI pipeline promoted an untested build to the production repository. The production Kubernetes cluster pulled and deployed it. The application is crashing.

Step-by-Step

Step 1 - Rollback immediately. Deploy the previous known-good version from the production repository. Kubernetes: kubectl rollout undo or update the image tag to the previous version.

Step 2 - Remove the bad artifact. Delete the incorrectly promoted artifact from the production repository (or mark it as blocked/quarantined if deletion is not allowed by retention policy).

Step 3 - Root cause. How was an untested artifact promoted? Was the promotion gate bypassed? Did the test stage pass erroneously? Was it a manual promotion without approval?

Prevention

Automated promotion gates: artifacts can only be promoted to production if they have passing test results attached (build info in Artifactory, properties in Nexus). Require manual approval for production promotion (Slack approval, Jira ticket, ArgoCD sync approval). Artifact signing - only CI-signed artifacts are accepted in the production repository. Kubernetes admission webhook rejects unsigned images.

Q42 Nexus/Artifactory disk full

What Happened

The artifact manager's storage is 100% full. Pushes fail. Some pulls may fail. CI pipelines are blocked. Developers cannot publish builds.

Step-by-Step

Step 1 - Emergency cleanup. Delete the largest, least valuable artifacts immediately. Docker images without tags (dangling manifests). SNAPSHOT artifacts older than 7 days. Build artifacts from deleted branches. Use the admin API for fast bulk deletion.

Step 2 - Compact blob store (Nexus). After deletion, run blob store compaction to reclaim disk space. Without compaction, deleted artifacts still occupy disk.

Step 3 - Add storage. Expand the disk (cloud: resize EBS volume or add a volume). Or migrate the blob store to S3/GCS for unlimited storage.

Step 4 - Implement retention policies. Create cleanup policies for every repository. Schedule automatic cleanup. Set monitoring alerts at 70% and 85% disk usage. See Q16 for retention policy design.

Q43 Vulnerability in base image used by 50 services

What Happened

A Critical CVE is announced in the ubuntu:22.04 base image. Your organization has 50 microservices all built on ubuntu:22.04. All 50 production images are vulnerable.

Step-by-Step

Step 1 - Assess impact. If using Artifactory Xray: impact analysis shows all 50 affected images instantly. If using Harbor: filter scan results by CVE across all projects. If manual: search for FROM ubuntu:22.04 across all Dockerfiles.

Step 2 - Rebuild with patched base. Update the base image to the patched version. Rebuild all 50 images. Push to the artifact repository. New scans should show the CVE is resolved.

Step 3 - Deploy. Priority: internet-facing services first, then internal services. Use automated pipelines - do not manually deploy 50 services.

Step 4 - Block the vulnerable base image. In the repository, mark the vulnerable tag as blocked. New builds that try to use the vulnerable base image fail with a clear message. Forces all teams to update.

Prevention

Use a curated base image that your organization maintains and patches centrally. All teams build FROM your-registry.example.com/base/ubuntu:22.04. When a CVE is found, patch the base image once - all downstream rebuilds pick up the fix. Automate nightly rebuilds of all images to pick up base image updates continuously. SBOM tracking (Q32) enables instant impact analysis.

Q44 Artifact registry credentials leaked

What Happened

A CI pipeline's registry credentials were committed to Git, posted in a Slack channel, or found in a public S3 bucket. An attacker could push malicious images to your registry.

Step-by-Step

Step 1 - Revoke the credentials immediately. Rotate the password, token, or API key. Do this within minutes - attackers scan for leaked credentials in real-time.

Step 2 - Audit recent pushes. Check the registry's audit log for any pushes from unauthorized sources during the exposure window. Compare image digests - was any image modified or replaced?

Step 3 - If any unauthorized push occurred: Quarantine affected images. Rebuild from known-good source. Notify teams using the affected images. Scan all affected images for malware.

Step 4 - Rotate all related credentials. If the leaked credential was a service account, all systems using that account need updated credentials.

Prevention

Never use long-lived credentials. Use OIDC/IAM-based authentication (AWS ECR uses IAM roles - no static credentials). Use short-lived tokens (Artifactory access tokens with expiry). Scan Git repositories for credential patterns (gitleaks in pre-commit hooks). Store credentials in CI secret managers, never in environment variables or config files.

Q45 Harbor replication lag - production pulling stale images

What Happened

A new image was pushed to the primary Harbor. Replication to the secondary Harbor (production region) is delayed. Kubernetes in the production region pulls the old image - deploying the previous version instead of the new one.

Step-by-Step

Step 1 - Check replication status. Harbor UI → Replications → check the job status. Is it pending, in progress, or failed?

Step 2 - If replication is slow (large image): Wait for completion. Monitor progress. Increase network bandwidth between sites if this is recurring.

Step 3 - If replication failed: Check error logs. Common causes: network timeout, authentication failure (expired token), destination disk full. Fix the root cause and retry.

Step 4 - Force immediate pull. If urgent, manually pull and push to the secondary: skopeo copy docker://primary-harbor/app:v2.1 docker://secondary-harbor/app:v2.1.

Prevention

Use event-based replication (triggers immediately on push) instead of scheduled (waits for the next schedule). Monitor replication lag as a metric - alert when lag exceeds 5 minutes. Reference images by digest (@sha256:...) in Kubernetes manifests - if the digest is not available in the local registry, the pull fails clearly instead of silently using an old tag. Include replication verification as a pre-deployment check: "Is the expected image digest present in the production registry?"

Interview Tips

For Junior Roles

Focus on Q1-Q14. Know the three repository types (hosted, proxy, group). Explain why artifacts should not be in Git. Know the difference between Nexus and Artifactory at a high level. Understand semantic versioning and the SNAPSHOT vs release distinction.

For Mid-Level Roles

Q1-Q32. Be ready to discuss CI/CD integration (how your pipeline publishes artifacts), artifact promotion (how an artifact moves from dev to production), vulnerability scanning (Xray, Trivy), and replication (multi-region). Know how to set up a Docker proxy to avoid rate limits.

For Senior Roles

All 45 questions. Supply chain security (SLSA, cosign, SBOM), architecture decisions (Nexus vs Artifactory vs Harbor), retention strategies, and cost optimization separate senior from mid-level. Be ready to design an artifact management architecture for a multi-region organization.

Quick Reference Cheat Sheet

Topic	Key Takeaway
Why not Git	Git is for source code (text, small). Artifact repos are for binaries (large, versioned, metadata).
Hosted/Proxy/Group	Hosted = your artifacts. Proxy = cached external. Group = unified URL for all.
Nexus	Free OSS. 20+ formats. Java-based. Good for budget-conscious teams.
Artifactory	Most feature-rich. Build info, promotion, Xray scanning. Enterprise choice.
Harbor	Container-only. CNCF graduated. Free. Trivy scanning. RBAC. Replication.
Cloud registries	ECR/GCR/ACR - use for single-cloud. Tightest IAM integration.
Immutability	Never overwrite a released artifact. Use digests, not :latest. Golden rule.
Promotion	Copy (not rebuild) artifacts through dev → staging → prod repos. Binary identity.
Proxy repos	Cache external deps. Build resilience. Avoid rate limits. Save bandwidth.
Scanning	Xray (Artifactory) or Trivy (Harbor). Continuous. Block vulnerable artifacts.
Replication	Multi-region. Event-based for low lag. DR standby. Edge distribution.
SBOM	CycloneDX/SPDX. Generated by syft/Trivy. Attached as OCI artifact.
Image signing	cosign (keyless, OIDC). Admission webhook enforces in K8s.
SLSA	Supply chain security framework. Levels 1-4. Provenance attestations.
Retention	Production: keep forever. Staging: 90 days. Snapshots: 30 days. Automate cleanup.
Rate limits	Docker Hub 100/6hr anonymous. Proxy repos eliminate the problem.
Cost	Cleanup automation + storage tiering + deduplication + smaller images.